Static Program Slicing for Quantum Programs

Anonymous Author(s)

***Abstract*—Quantum programs are increasingly developed in general-purpose languages, particularly Python, using libraries such as Qiskit to construct quantum circuits. Yet tools that help developers understand and debug these programs remain limited. Program slicing identifies the statements that can affect a value of interest, but applying it to quantum software requires tracking dependencies through gates and measurements, together with the classical computations that determine which operations act on which qubits. In this paper, we extend the srcML infrastructure and the srcSlice slicer with a quantum layer that identifies the quantum role of each API call, reconstructs the circuit a program builds without executing it, and slices every qubit, including loops that run on the quantum device. Each slice identifies relevant source lines, the loop iterations that contribute circuit operations, and the classical statements that determine those operations. We implement the approach in a tool, QuSlice, and evaluate it on GitHub programs, Bugs4Q, the Qiskit textbook, Qiskit HumanEval and QASMBench. The reconstructed circuits match those Qiskit builds, and the slices match reference slices on 15 of 16 programs, where QuSlice is the more precise on the remaining one. The classical statements are complete for 90% of qubits, and for 86% in programs not used during development. QuSlice slices 4,182 of 7,568 GitHub programs in a median of 131 ms each and a circuit of 5.8 million operations in 10 s after a 12-minute markup, and a median slice holds 24% of a program's code lines. These results show that static slicing can support the comprehension of quantum programs, pointing developers to the statements, loop iterations and classical computations that explain a qubit's state.**

***Index Terms*—program slicing, static analysis, quantum software engineering, Qiskit, srcML, program comprehension**

I. Introduction

Most quantum programs are not written as circuits. They are Python programs that build a circuit with a library such as Qiskit [1][1], deciding in loops, functions and computed indices which gates act on which qubits. When a qubit ends in an unexpected state, the developer of such a program needs the statements that influence it, and in a quantum program these are harder to find by reading than in a classical one. A controlled gate can change the qubit that controls it, and a measured bit can steer operations several lines and loop iterations later. Quantum developers report that the methods they debug with, printing, drawing circuits and reading code, do not scale [2].

Program slicing gives a developer exactly these statements for classical programs [3], and srcSlice[2] computes them statically, from the source and without running the program, for large systems on the representation of their source by srcML[3] [4], [5]. For a quantum program, however, a classical slicer sees only a circuit object that every gate call uses, and the tools that do understand circuits, such as Qiskit's DAG and its `LightCone` pass [6], need the program to run first and lose the link from each gate to the line and iteration that added it. The analyses that read quantum source code look for known patterns [7], [8], and a qubit slicer on srcML has been named as future work [9].

This paper presents an approach to static slicing of quantum programs. It extends srcML and srcSlice with a quantum layer that reads the quantum role of each call, reconstructs the circuit a program builds, over-approximating what cannot be known without running it, slices every qubit of that circuit along the gates and measured bits that can affect it, and shows each slice as source lines, the loop iterations that put them there and the classical statements that compute them. We implement the approach in a tool, QuSlice, for Qiskit programs and for OpenQASM 2 and 3. For OpenQASM, the tool produces the srcML markup that Behler et al. specified [10] and that had so far been written by hand, and builds on it the slicer they proposed [9].

The paper makes four contributions. The first is the approach, a semantic layer over srcML and srcSlice with five extensions, whose over-approximations can only enlarge a slice, applied to Qiskit and, through automatic front ends for Behler et al.'s markup, to OpenQASM 2 and 3. The second is a way to slice loops that run on the device, with an argument that it is sound. The third is a qubit profile, the quantum counterpart of srcSlice's slice profile. The fourth is an evaluation on 7,568 GitHub programs, Bugs4Q [11], the Qiskit textbook [12], Qiskit HumanEval [13] and QASMBench [14]. Without running a program, QuSlice reconstructs the circuit Qiskit builds on all 16 of our runnable programs, and its slices match the light cone of that circuit in 15 of them. It traces 4,182 GitHub programs, takes 131 ms per program in the median, slices every qubit of a circuit of 5.8 million operations in 10 s after marking it up in 12 minutes, and points a developer to 24% of a program's code lines.

Section II works through an example, Section III reviews related work, Section IV presents the approach, Sections V and VI the evaluation, and the last two sections discuss its limits and conclude. The tool, the data and the scripts are in an anonymous replication package.[TODO: anonymous package link for Paper B]

II. A Motivating Example

Quantum programs in Qiskit: A Qiskit program does not compute with quantum states directly. It builds a circuit, a sequence of operations on numbered qubits, by calling methods of a `QuantumCircuit` object, and hands the circuit

[1]https://github.com/Qiskit/qiskit
[2]https://github.com/srcML/srcSlice
[3]https://www.srcml.org

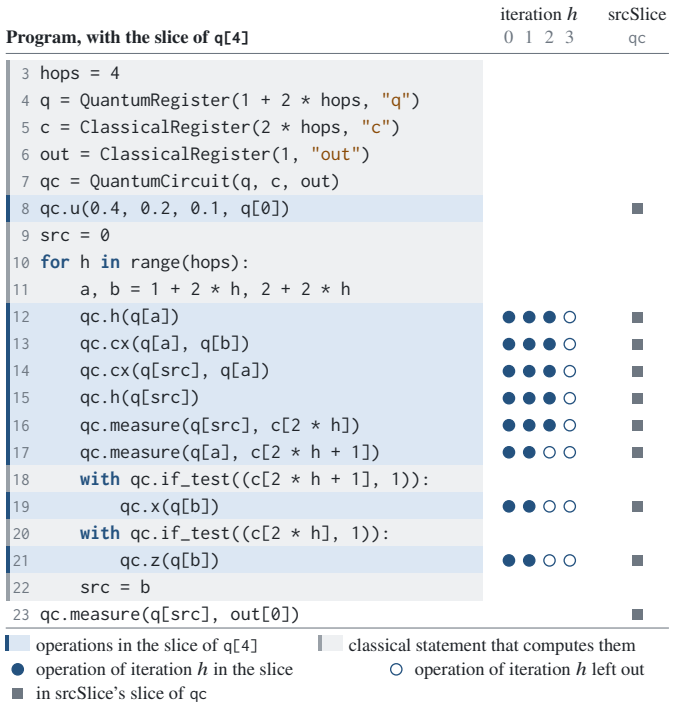


Fig. 1. A chain of four teleportation hops, our own program, with the imports on lines 1 and 2 omitted. Shading gives the slice of `q[4]` that QuSlice computes without running the program, blue for the lines whose operations are in it and grey for the statements that compute them. The dots give, for each gate line of the loop, the iterations h whose operation is in the slice. srcSlice's slice of `qc` is every gate line.

to a simulator or a quantum device to run. A gate such as `h` or `x` changes the state of the one qubit it acts on. A two-qubit gate such as the controlled NOT, `cx`, acts on a control and a target, and although it is usually described as flipping the target when the control is 1, it can change the state of both qubits, its control included. A measurement reads a qubit into a classical bit and disturbs the qubit as it does so. An `if_test` block conditions the operations inside it on such bits while the circuit runs, so a measured value can steer operations that come later. What a qubit's final state depends on is therefore decided by the gates on it, the gates it shares with other qubits and the bits that condition them, and in the program all three are chosen by ordinary Python, through its loops, functions and computed indices.

The example: Fig. 1 shows a Qiskit program that teleports the state of `q[0]` along a chain of qubits. Each hop h prepares an entangled pair `q[a]`, `q[b]` (lines 12 and 13), measures the current carrier `q[src]` and `q[a]` (lines 14 to 17), and repairs `q[b]` with gates conditioned on the two measured bits (lines 18 to 21). The carrier then moves to `q[b]`, so the variable `src` names a different qubit on every iteration, and the qubit indices on lines 11 and 22 are computed from the loop counter.

Suppose the state after the second hop, carried by `q[4]`, looks wrong. A developer who reads the program has to work out which iterations act on `q[4]`, follow `src` through two reassignments, and remember that a measured bit steers the repair on lines 19 and 21. A classical slicer does not help. srcSlice reads `qc` as a single variable that every gate line uses, so its slice of `qc` is every gate line, and nothing in it distinguishes one hop from another or one qubit from another.

The slice of `q[4]` that QuSlice computes keeps 22 of the 34 operations. It keeps the first two hops in full and, of the third, the gates on lines 12 to 16, and it names these iterations. The third hop's preparation of `q[5]` and `q[6]` is kept although neither qubit is `q[4]`, because the CNOT of line 14 in that hop has `q[4]` as its control and can change it. The slice also keeps the statements that compute the indices and the carrier, lines 9 to 11 and 22, and leaves out line 23, which measures another qubit. None of this needs the program to run, so it works on programs that cannot be run, such as fragments or programs for an older Qiskit release.

III. Background and Related Work

This section reviews the infrastructure the approach builds on, the analyses that already read quantum source code, the tools that work on circuits and the dependence models proposed for slicing, and ends with the gaps that Table I summarizes.

A. srcML and srcSlice

srcML represents source code as XML in which the syntax of the language is marked up while the original text, comments and layout included, is kept, so the XML converts back to the exact source [15], [5]. Because srcML parses without compiling, it accepts fragments and programs whose libraries are missing, which suits code collected from repositories. srcSlice computes slices from srcML for every variable of a system at once [4]. Instead of building a program dependence graph [16], [17], it keeps a slice profile per variable, the lines that define and use it, the variables it flows into and the calls that receive it, and slices from these profiles, which let it process the Linux kernel. Neither tool knows what a quantum program means. To srcML a Qiskit program is Python, and to srcSlice its circuit is a single variable that every gate call uses, so the slice of any qubit is every line that touches the circuit.

B. Analyses of quantum source code

Behler et al. specified a srcML markup for OpenQASM 3 [10] and built QStatic[4] on it, which derives the order in which a program applies operations to each qubit, unrolling loops and expanding gate definitions, and uses that order to find iteration patterns and suggest refactorings [9]. They demonstrate the markup on the 16 example files of OpenQASM 3, which they mark up by hand since the markup has no released parser, leave `while` loops for later, and name a slicer that finds the lines affecting a qubit as future work. Kaul et al. translate Qiskit and OpenQASM programs into a code property graph, the QCPG, and query it for bugs and code smells [18]. LintQ[5] runs CodeQL queries over Qiskit

[4]https://github.com/srcML/QStatic
[5]https://github.com/sola-st/LintQ

programs, unrolling loops for up to ten iterations to relate their operations, and was applied to 7,568 programs from GitHub [7]. QChecker checks Qiskit programs against bug patterns over their syntax trees [8], Q-PAC is another pattern-based detector [19], Chen et al. define smells specific to quantum code [20], and Xia and Zhao analyze entanglement statically [21]. These analyses look for known patterns and properties. None returns the statements that determine a qubit.

C. Tools that work on circuits

Once a program has run and built its circuit, Qiskit can list the operations that precede a qubit's last operation as the ancestors in its DAG representation, and its `LightCone` pass keeps the gates that can affect given qubits or an observable and drops those that commute with it [6]. Qiskit's slicing utilities split a circuit into consecutive pieces at barriers, by depth or by gate type [22], and the Cirquo tool of Metwalli and Van Meter splits a circuit at barriers and maps its gates back to source lines through tracebacks taken while the program runs [23], [24]. Working on the circuit has two costs for a developer. The program must run, with its Qiskit version, its inputs and sometimes access to hardware, and the circuit no longer says which line and which loop iteration added each gate.

D. Dependence models for quantum programs

Yu and Zhao describe a dependence graph for quantum programs whose edges add four quantum kinds to data and control dependence, and outline slicing on it without implementing it [25]. Compilers order and place gates by their backward light cone, the gates whose effect can propagate to given qubits by the end of a circuit [26], [27]. The approach slices by this light cone, followed further from each measurement to the operations its bit conditions. Two roadmaps point to the same need, one describing dependence graphs as an aid to understanding quantum programs [28], the other finding that dependence analysis of circuits has hardly been studied [29], and developers report that their debugging methods, printing, drawing circuits and reading code, do not scale [2].

E. Gaps

Table I compares these approaches with QuSlice. Three gaps remain. No slicer reads Qiskit, the language most quantum programs are written in, and the slicer named for OpenQASM 3 is future work. The tools that slice or prune circuits need the program to run and lose the link to its lines and loop iterations. The analyzers that read Qiskit source look for patterns, and none has been evaluated as a slicer on real programs.

IV. The Approach

The approach adds a quantum layer to srcML and srcSlice. srcML parses the program, and five extensions, E1 to E5, give its markup a quantum meaning: they read what each call does to which qubits (E1), reconstruct the circuit the program builds (E2), slice every qubit of that circuit (E3), add the classical statements that compute each slice (E4), and summarize each qubit in a profile (E5). The result is shown on the program's lines. We implement the approach in a tool, QuSlice, for Qiskit programs [1] and OpenQASM 2 circuits. Fig. 2 shows its parts and where each comes from, and Fig. 3 follows the program of Fig. 1 through them.

A. Why srcML and srcSlice

A slicer whose output a developer reads needs four properties that srcML and srcSlice already have. srcML parses a file as text and never compiles or runs it [15], so the approach accepts fragments, programs written for Qiskit releases that no longer install, and code from sources one would not execute, and it needs neither a simulator nor a quantum device. srcML keeps every character of the source, layout and comments included, with the position of each element, and converts its markup back to the exact text [15], so a slice can be shown on the developer's own lines. srcML marks up all its languages with one vocabulary [5], and a markup for OpenQASM has been specified [10], so a new front end needs only its quantum layer and the classical and quantum parts of a program can share one representation. Last, srcSlice slices from per-variable profiles that it computes on demand rather than from a dependence graph, which let it process the Linux kernel in about 20 minutes with accuracy comparable to CodeSurfer [4]. Behler et al. proposed adapting srcML's slicer to find the lines that affect a qubit [9], and the approach carries out that proposal for Qiskit.

B. The quantum layer on srcML

Qiskit is ordinary Python, so unlike OpenQASM it needs no new markup, but its quantum meaning lies in library calls that no markup captures. srcSlice sees the program as variables. Its profile of `qc` uses every gate line, because every gate is a call on the one circuit object, and its profile of the register `q` records that it reaches both arguments of `cx` on line 14. srcSlice therefore knows which variables reach which argument of which call, but not which qubits a gate acts on, so its slice of `qc` is the whole program and no qubit has a slice at all.

The quantum layer supplies that meaning statement by statement, as Behler et al.'s analyzer does for OpenQASM [10]. srcML gives each statement's element, its nested blocks and its position. The layer walks these elements in the order the program runs them, keeps the value of every variable it can know, and evaluates the expressions inside a statement from their text, which srcML keeps character for character. Table II lists what it does with each element, what srcSlice does with the same element, and where Fig. 1 uses it. The subsections that follow describe each extension.

C. E1, quantum roles of calls

Qiskit applies a gate through a method of the circuit, as in `qc.cx(a, b)` or `qc.ry(theta, q)`. E1 decides, for every such call, which arguments are qubits, which are classical bits and which are angles, and which leading qubits

TABLE I
APPROACHES THAT ANALYZE QUANTUM PROGRAMS OR CIRCUITS, AGAINST WHAT A SOURCE-LEVEL QUANTUM SLICER NEEDS

Approach	Reads Qiskit	Without running	Slice per qubit	Measured bits	Loops	Source lines	Evaluated on
srcSlice [4]	○	●	○	–	●	●	Linux kernel
Behler et al., QStatic [10], [9]	○	●	○	◐	◐	●	16 OpenQASM 3 example files
QPDG, Yu and Zhao [25]	●	●	●	●	?	●	5 listings, not implemented
QCPG, Kaul et al. [18]	●	●	○	●	●	●	3 listings, 1 Qiskit/OpenQASM circuit
LintQ [7]	●	●	○	●	◐	●	7,568 GitHub programs, 216 problems found by inspection
QChecker [8]	●	●	○	–	–	●	42 Bugs4Q bugs
Qiskit DAG, `LightCone` [6]	○	◐	●	○	○	○	–
Qiskit circuit slicing [22]	○	◐	○	–	–	○	–
Cirquo, Metwalli and Van Meter [23], [24]	●	○	○	–	–	◐	1 Grover case study, 1 user survey
QuSlice	●	●	●	●	●	●	7,568 GitHub, 151 Bugs4Q, 72 textbook and 151 Qiskit HumanEval programs; 169 QASMBench circuits

● has the property, ◐ has it in part, ○ lacks it, ? means the authors do not specify it, and – means it does not apply. *Reads Qiskit*, the approach takes Qiskit programs as input. *Without running*, it analyzes the program without executing it; circuit tools need the program run to obtain the circuit, unless the circuit is given in OpenQASM. *Slice per qubit*, it returns the operations or lines that can affect a chosen qubit. *Measured bits*, it follows a measured bit to the operations the bit conditions. *Loops*, it follows loops of any length. *Source lines*, it reports results on the program's lines. *Evaluated on*, the data of the original evaluation.

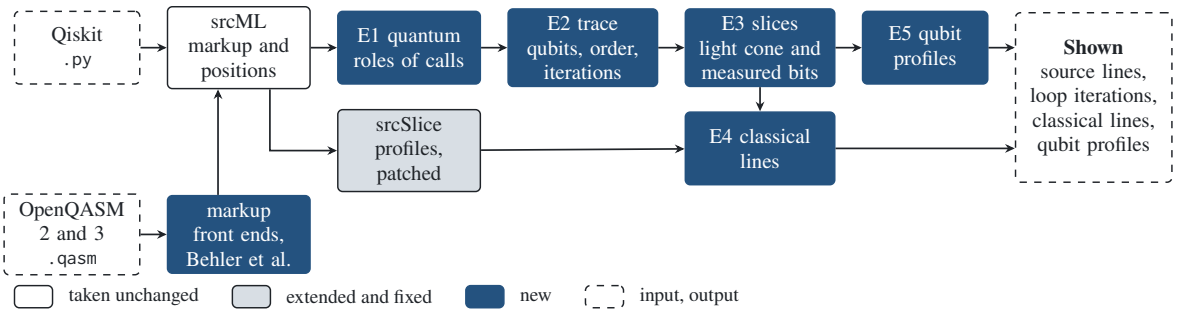


Fig. 2. The parts of the approach. A Qiskit program is marked up by srcML, and an OpenQASM 2 or 3 program by front ends that produce Behler et al.'s markup. The extensions E1 to E5 read both, and srcSlice, patched, gives the classical lines of a Qiskit program. What the developer is shown is on the right.

are controls. It reads this from a role table derived from the signatures of Qiskit's `QuantumCircuit` methods rather than from a hand-written list, so the table follows the library. It holds the 47 methods of Qiskit 2.3 that take qubits and add an operation, 15 methods that Qiskit 1.0 removed but older programs still call, and six methods whose arguments the signatures do not name uniformly, 68 in all. When E2 evaluates a call whose receiver is a circuit, the table turns it into a gate on qubit operands, as the gate and qubit colours mark in stage 1 of Fig. 3. The table plays the part that the quantum argument lists of the OpenQASM markup [10] play in a language whose syntax states the roles. Table III lists the methods by kind, with the calls outside the table and how they are read. A circuit composed into another is mapped onto the qubits it is given, and a gate the table does not know, such as a circuit from Qiskit's library, becomes one operation on all its qubits, which can only enlarge a slice. A method the approach does not model is reported to the developer, never skipped silently.

D. E2, the static trace

Qiskit computes qubit indices, register sizes and angles in Python, so E2 interprets the program over srcML's statements, as Section IV-B describes, and records the operations the program would append, as a trace, much as the execution-order analysis of Behler et al. does for OpenQASM 3 [9]. Each operation carries its source line, from the position srcML records (the grey `pos:start` in Fig. 3), and the iteration of every loop around it. The interpretation unrolls loops of the host program of up to 10,000 iterations, inlines its functions, follows its branches, and reads `if_test` and `c_if`, Qiskit's two ways of conditioning gates on measured bits (the bit and condition colours), as stage 2 of Fig. 3 shows. Where a value cannot be known without running the program, it over-approximates. An index it cannot evaluate acts on

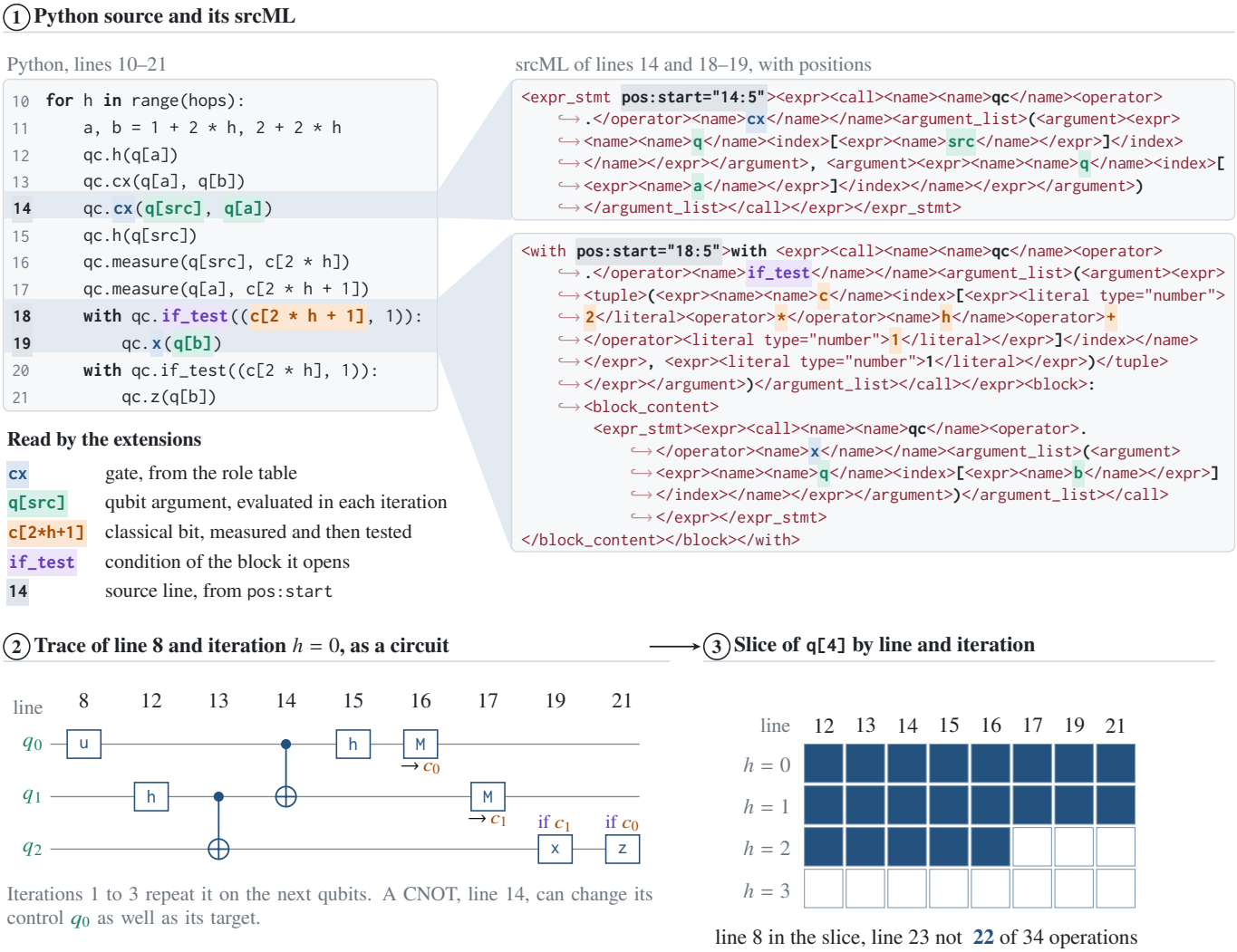


Fig. 3. The approach on the program of Fig. 1. (1) The loop of the program and srcML's markup of line 14 and of lines 18 and 19, each band joining a line to its markup. srcML keeps every character of the source, shown in bold among the tags, and the elements the extensions read carry the same colour on both sides, as the key under the source explains. (2) The trace of line 8 and the first iteration drawn as a circuit, each operation under the line that appends it. (3) The slice of `q[4]`, one cell per operation of a loop line in each iteration, filled when the operation is in the slice.

every qubit it may denote, both branches of a condition it cannot decide are followed, and a loop of unknown length is repeated as Section IV-F describes, up to 65 copies. Each over-approximation adds operations or qubits, so it can only enlarge a slice, and each is reported to the developer, as is a circuit passed to code the interpretation does not model.

E. E3, the slices

To srcSlice, `qc.cx(a, b)` defines no variable and uses `qc`, `a` and `b`. E3 reads each operation of the trace by its qubits and bits instead. A gate writes every qubit it acts on, its controls included, a measurement writes its bit, and a conditioned operation reads the bits it tests. A qubit's slice is then the backward light cone of its final state, the relation compilers use to order and place gates [26], [27], followed from each measurement to the operations its bit conditions. E3 computes it in one backward pass over the trace with a set of needed qubits and bits, in time linear in the trace, skipping barriers and the measurements at the end of the circuit. An operation that may act on several qubits adds all of them to the needed set, and a bit leaves the set only when an operation certainly writes it, so the over-approximations of E2 stay sound. Like srcSlice, E3 computes dependence while it slices and builds no graph. The pass returns the operations of the slice with their lines and iterations, as stage 3 of Fig. 3 shows, and the forward pass from an operation gives its forward slice. For large circuits, the pass carries the set of criterion qubits on each needed location and so slices every qubit at once.

F. Loops that run on the device

Recent Qiskit releases let a circuit contain its own loops and branches, `while_loop`, `for_loop`, `switch` and the `else` branch of `if_test`, which run on the device and test measured bits. A `for_loop` over a known range is unrolled.

TABLE II
WHAT THE QUANTUM LAYER DOES WITH EACH SRCML ELEMENT OF A QISKIT OR OPENQASM PROGRAM, AGAINST SRCSLICE

srcML element	srcSlice	The quantum layer	Ext.	Fig. 1
`<expr_stmt>`, an assignment	defines the target, uses the other names	evaluates the value, such as a register size, an index or a circuit, and records it	E2	3–7, 9, 11, 22
`<expr_stmt>`, a call on a circuit	uses the circuit and the argument names	looks the method up in the role table and appends one operation on the qubits its arguments denote; a measurement also writes its bit	E1, E2	8, 12–17, 19, 21
`<for>`	reads the body once	unrolls the loop, up to 10,000 iterations, and tags each operation with its iteration; a loop of unknown length is repeated	E2	10
`<while>`	reads the body once	runs while the condition is known, then repeats the body as a loop of unknown length	E2	–
`<if_stmt>`	reads every branch	follows the branch the condition selects, or every branch when it is unknown	E2	–
`<with>` around `if_test`, `switch` or a device loop	reads the block as ordinary code	conditions every operation of the block on the bits it tests; device loops are repeated, Section IV-F	E2, E3	18–21
`<function>`, `<return>`	links parameters to the arguments of each call	inlines the function at each call with its arguments and returns its value	E2	–
`<class>`, `<import>`, `<try>`	reads them as declarations	registers methods, resolves import aliases, reads the `try` block	E2	–
`pos:start` of every element	line numbers of its profiles	the source line of every operation	E2	all
OpenQASM 2, in Behler et al.'s markup, produced by the front end of Section IV-I				
`<decl_stmt>`, `qreg` or `creg`	–	allocates the qubits or bits; the declaration is a classical line of every operation on them	E2, E4	–
`<include>`, `<gate>`, `<gate_decl>`	–	role-table entries, read from the markup of `qelib1.inc` or from the definition	E1, E4	–
`<call>` with a quantum `<argument_list>`	–	one operation per qubit tuple, expanded over the registers it names	E1, E2	–
`<measure>`, `<reset>`, `<barrier>`	–	writes the bits, resets the qubits; a barrier is never sliced	E2	–
`<if_stmt>`, `if(c==n)`	–	conditions the operation on every bit of the register	E2, E3	–
OpenQASM 3, the same elements and these				
`<for>`, `<while>`, `<if_stmt>`, `<switch>`	–	over known values, unrolled or decided; over measured bits, conditioned on them, and a `while` repeated	E2, E3	–
`<specifier>` of a `<call>`	–	`ctrl` and `negctrl` add control qubits to the operation	E1	–
`<unsupported>`	–	`def`, `let`, `delay`, calibration, reported; a subroutine call is one operation on its qubits	E2	–
the operations appended, with their bits	–	slices every qubit along the light cone and the measured bits	E3	8, 12–17, 19, 21
the variables of the sliced lines	profiles, patched for Python	follows the profiles back to the statements that compute the slice	E4	3–7, 9–11, 18, 20, 22
the operations on each qubit	–	gathers the qubit's uses, names, dependents, bits and slices	E5	–

Any other device loop is repeated, with every operation of its body conditioned on the bits its condition reads, until a repetition adds no qubit or bit to the needed set.

Proposition 1: Let a device loop with body B end with probability one, let Λ be the qubits and bits B acts on, and let the slice be computed on $|\Lambda|+1$ conditioned copies of B. Deleting from B every operation that none of its copies puts in the slice of W leaves the state of W unchanged for every input.

Proof sketch: A loop that runs at most K times behaves as K copies of B, each conditioned on the bits that decide the loop. In the backward pass every copy is processed in the same way and the needed set only grows. Once a copy adds nothing, every earlier copy keeps the same operations, and the needed set can grow at most $|\Lambda|$ times. So for every K the copies keep only operations that one of the $|\Lambda|+1$ copies keeps. The bits that decide the loop are read by every copy, so the measurements that write them stay in the slice and the distribution of the number of iterations is unchanged. A loop that ends with probability one is the limit of its bounded prefixes. ∎

G. E4, the classical statements

The statements that compute a slice's qubits, angles, conditions and loop bounds come from srcSlice's profiles. E4 starts from the slice's lines and the headers of the loops, conditions and `with` blocks around them. Every profile used on a start line is taken, and the closure adds its definition

TABLE III
HOW E1 READS THE CALLS OF A QISKIT PROGRAM

Calls	Methods	Read as	Programs
In the role table			
Single-qubit gates, `h`, `rz`, `u`	18	one operation on its qubit	4,413
Two-qubit gates, `cx`, `cz`, `swap`	20	one operation on both qubits	2,933
Multi-qubit gates, `ccx`, `mcx`	8	one operation on all its qubits	718
Removed in Qiskit 1.0, `cnot`, `u3`	15	as gates of the same arity	1,033
`measure`, `reset`, `barrier`, ...	6	a measurement writes its bit; a barrier is never sliced	3,701
Outside the role table			
`append`, `compose`, `measure_all`	4	mapped onto the target's qubits	3,509
`if_test`, `while_loop`, `switch`, ...	5	conditioned, Section IV-F	169
`inverse`, `power`, ...	6	result unknown, reported	623
`circuit_to_dag`	1	trace of that circuit ends	100
any other, `reverse_bits`, `run`, ...	–	not modelled, reported	183

Programs counts the programs of LintQ's corpus whose source calls a method of the row, and for the last row those in which the interpretation meets a method it does not model.

lines, every profile whose dependent variables name it, and, for a parameter at position k of a function f, every profile passed to f at position k. srcSlice's profiles record forward facts, which variables a variable reaches and where it is passed, and E4 follows them backward from the start lines. srcSlice runs with a patch that adds Python, covering keyword arguments, the receiver offset of methods, loop, `with`, tuple and comprehension targets, nested scopes and f-strings, and with fixes that apply to every language srcSlice supports. Two of these fixes came from this work. srcSlice recorded no use of a variable inside a subscript, as in `q[a]`, which is how Qiskit names a qubit and how C indexes an array, and it missed a module variable read inside a function. The patch applies to the public srcSlice and comes with regression tests.

H. E5, qubit profiles

srcSlice does not store a slice as a list of lines. It summarizes every variable in a *slice profile* and reads slices off these profiles [4]. For each variable, srcSlice reports the lines that define it, `def`, and use it, `use`; the variables whose values are computed from it, `dvars`; its aliases, `pointers`; and the functions it is passed to, `cfuncs`, each with the position of the argument, so that `cx{2}` records that the variable is the second argument of a call to `cx`. The original paper names the same fields `slines`, `dvariables`, `pointers` and `cfunctions`. The slice of a variable is the closure of its lines over these fields, through its dependent variables, its aliases and the parameters of the functions it is passed to. E5 gives each qubit a profile of the same kind.

TABLE IV
SRCSLICE'S PROFILE OF THE REGISTER `q` AND QUSLICE'S PROFILE OF THE QUBIT `q[4]`, BOTH COMPUTED ON THE PROGRAM OF FIG. 1

Entry, srcSlice field	srcSlice, variable `q`	QuSlice, qubit `q[4]`
Lines `def`, `use`	defined on 4, used on 7, 8, 12–17, 19, 21, 23	13–16, 19, 21, each operation reading and writing `q[4]`
Flows into `dvars`	the variable `qc`	the qubits `q[2]`, `q[3]`, `q[5]`–`q[8]`, which its first operation can affect
Other names `pointers`	none	`q[b]` in iteration 1, `q[src]` in iteration 2
Passed to `cfuncs`	`QuantumCircuit{1}`, `u{4}`, `h{1}`, `cx{1}`, `cx{2}`, `measure{1}`, `x{1}`, `z{1}`	no function of the program
Bits	–	`c[4]`, which conditions line 21
Slice	every line that uses `q` or `qc`	backward 8, 12–17, 19, 21 and forward 13–17, 19, 21, 23, each with its iterations

Definition 1 (Qubit profile): The profile of a qubit q records its *uses*, the lines of the operations on q; its *names*, the source expressions that denote q; its *functions*, the call sites of the functions in which q is acted on; its *dependents*, the qubits that the first operation on q reaches; its *bits*, those measured from q, with the lines they condition; and its *backward* and *forward* lines, those of its slice and of the forward slice of its first operation.

Table IV sets the two side by side on the program of Fig. 1. To srcSlice the register `q` is one variable, used on every gate line and passed to every gate, as its `cfuncs` show, so its profile cannot tell one qubit from another. The profile of `q[4]` keeps only the six lines whose operations act on it. Its two names show at once that `q[4]` is first the target of a hop, as `q[b]`, and then the carrier, as `q[src]`. Its bit `c[4]` points to the conditioned gate on line 21, which a variable profile has no entry for. E5 computes the profile from one backward and one forward pass over the trace and from the srcML of the gate calls on the qubit.

I. Front ends for Qiskit and OpenQASM

Qiskit programs, the language of Bugs4Q and of LintQ's corpus [11], [7], are parsed by srcML as Python. srcML 1.1 has no OpenQASM language. Behler et al. specified a markup for it but released no parser, so their analyses run on files marked up by hand [10], [9]. QuSlice produces that markup automatically for OpenQASM 2, the format that Qiskit exports and benchmark suites ship in. Its front end parses the whole grammar of the language [30] and writes srcML in srcML's own format, every character kept and every element positioned, with Behler et al.'s elements, `<decl_stmt>` for registers, `<gate>` for definitions, `<call>` with a quantum argument list for gate applications, and `<measure>`, `<reset>`, `<barrier>` and `<if_stmt>`. The quantum layer reads this markup as it reads the markup of a Qiskit program, as the lower part of Table II lists. E1 takes the

roles of the standard gates from the markup of `qelib1.inc` itself and those of other gates from their definitions, and E2 expands each application over the registers it names, as the specification defines. E3 and E5 apply unchanged. E4 gives the declarations, gate definitions and conditions that a slice needs, since srcSlice has no OpenQASM support and OpenQASM 2 has no other classical statements. The front end also checks the rules the specification states beyond its grammar, such as declared names, register bounds, gate arity and the sizes in a broadcast, and reports a program that breaks one instead of slicing it.

OpenQASM 3 adds what Qiskit computes in Python and on the device, classical variables, loops, branches, switches, subroutines and gate modifiers [31]. QuSlice reads the part of the language that decides which operations act on which qubits under which conditions, declarations, assignments, gate definitions, gate calls with the `ctrl`, `negctrl`, `inv` and `pow` modifiers, both forms of measurement, `reset`, `barrier`, `box`, `if`, `for`, `while` and `switch`. A second front end parses these by the official grammar and marks them up with Behler et al.'s elements, `<for>`, `<while>` and `<if_stmt>` among them, and the layer applies E2 as it does to a Qiskit program. A loop or branch over values it knows is unrolled or decided, while one whose condition reads measured bits runs on the device, so its operations are conditioned on those bits and a `while` is repeated as Section IV-F describes. The remaining statements, `def`, `let`, `delay` and calibration, are kept in the markup and reported. A call to a subroutine is read as one operation on its qubit arguments, the only qubits a subroutine can reach, and an operand named through an alias acts on every qubit it may denote.

J. What the layer does not read

The approach slices what it can reconstruct. A program that loads its circuit from a file or builds it in code the interpretation does not follow yields no slice, and a method outside the role table stops the trace of its circuit, both reported with their cause. Over-approximation keeps slices sound but can make them larger than needed, for instance when an index depends on input the program reads at run time. The classical statements are the least exact part, since srcSlice does not link a value returned from a function to its call, which RQ1 measures. In OpenQASM 3, the bodies of subroutines and the targets of aliases are not read but over-approximated, and calibration code is left out.

V. Evaluation Design

The evaluation asks four questions about the approach as a developer would use it.

- **RQ1, faithfulness.** Does the approach compute, without running the program, the circuit the program builds, the slices of that circuit, and the classical statements the slices need?
- **RQ2, coverage.** How many real programs can it slice, and why does it fail on the others?
- **RQ3, cost.** How fast is it, and how does it scale against tools that work on the circuit?
- **RQ4, what a slice shows.** How much of a program does a slice ask a developer to read, and what does it add to the warnings of a bug detector?

Every analysis was planned in a protocol and changed only through dated deviations, which the replication package contains, and all of them are exploratory.

A. Subjects

Table V lists the programs and circuits. Bugs4Q [11] and the corpus that LintQ's authors collected from GitHub [7] are real programs, most written for Qiskit releases before 1.0. Qiskit HumanEval [13] holds tasks solved by Qiskit experts for current releases, and the Qiskit textbook [12] programs teach the standard algorithms. We wrote three sets ourselves, the 16 runnable programs used to check the trace, programs in which a size constant grows the circuit while the source stays fixed, and programs with loops that run on the device. The OpenQASM 2 front end is checked on the 1,212 OpenQASM 2 files up to 5 MB of MQT Bench [32], QASMBench and VeriQBench [33], and on test programs for the grammar and the specification's examples. The OpenQASM 3 front end is checked on the official grammar tests, on the 21 example programs of the language, on the OpenQASM 3 files of VeriQBench, and on programs that Qiskit's exporter writes from our programs and from the corpus files. The OpenQASM 2 front end is timed on the medium and large circuits of QASMBench [14].

B. References for faithfulness

For RQ1, our runnable programs are also run with Qiskit, and the trace must equal the circuit Qiskit builds, operation by operation. The slices must equal the backward light-cone slices of that built circuit, which the study computes with code separate from the tool, and on the Bugs4Q bugs that port to circuits, the line slices must equal those obtained by recording which line appends each operation while the port runs. The OpenQASM 2 front end's trace must equal the circuit Qiskit loads from the same file, on every OpenQASM 2 file of the corpora, it must reject exactly the files Qiskit rejects, and its srcML must give back the file byte for byte. Exported to OpenQASM 3, each of our programs must give the trace of its Python source, with the same conditions and may-operations, and each corpus file the circuit Qiskit loaded before exporting it. For the programs with device loops, an exact density-matrix simulation that branches over measurement outcomes deletes every operation outside a qubit's slice from the source and checks that the qubit's state is unchanged at six inputs, and a negative control reads each device loop's body once. The classical statements of E4 are compared with the values E2 tracks along the one path it follows, which give a second, flow-sensitive set of lines, on our programs and Bugs4Q, on a sample of LintQ's corpus, and on a held-out sample of 404 files of that corpus that were not inspected while the srcSlice

TABLE V
PROGRAMS AND CIRCUITS OF THE EVALUATION

Subject	Count	Traced	Qubits	Operations	Code lines	Used for
LintQ's corpus, GitHub programs	7,568	4,182	1–4,096	8 / 200,000	58	RQ1 to RQ4
Bugs4Q programs	151	111	1–20	4 / 84,000	15	RQ1, RQ2
Qiskit textbook	72	60	1–13	8 / 4,204	90	RQ2
Qiskit HumanEval	151	78	1–11	3 / 122	9	RQ2
Parametric programs, 5 sizes each, ours	4	18 of 20	9–10,000	2,980 / 181,200	12	RQ3
Programs with device loops, ours	10	10	3–4	18 / 37	12	RQ1
QASMBench medium and large, OpenQASM 2	169	164	11–10,000	550 / 5.8 million	–	RQ3

Traced counts the programs from which QuSlice reconstructs a circuit, the sizes of the parametric programs, and for QASMBench the circuits in valid OpenQASM 2. Qubits and operations describe the traced circuits, as smallest to largest and as median / largest, and 200,000 operations is the trace limit. Code lines exclude blank and comment lines and are medians. Programs by others are read and never run.

fixes were made. We fixed a threshold of 95% containment before the comparison.

C. Coverage, cost and what a slice shows

For RQ2, QuSlice traces every program, and the programs that yield no circuit are classified by a fixed rule on their text. For RQ3, QuSlice slices every qubit, up to 64, of each program of LintQ's corpus that builds a circuit, with a limit of 120 s per program. On the QASMBench circuits it slices every qubit in one pass, against three tools that slice one qubit of the built circuit, the ancestors in Qiskit's DAG, Qiskit's `LightCone` pass and a reimplementation of the QPDG [25], with a limit of 600 s per circuit for the baselines and 3,600 s for QuSlice, whose srcML markup of the largest circuits takes minutes. On the parametric programs, a QFT, a GHZ preparation, a QAOA ring and a repetition code, a size constant grows the trace while the source stays the same. All times are wall-clock on one machine, an Intel Xeon W-2140B at 3.2 GHz with 32 GB of memory, with Python 3.12 and Qiskit 2.3. For RQ4, we measure the share of a program's code lines that a qubit's slice holds, and compare QuSlice with LintQ on LintQ's public data, without running LintQ. For each warning of LintQ's five analyses that concern qubits and bits, QuSlice traces the program and evaluates a predicate on its qubit profiles, fixed before the comparison, at the warned line, on the warnings LintQ's authors labelled true and false positives and on all warnings of these analyses. The examples in the papers of LintQ and QChecker [7], [8] are transcribed as programs and sliced.

VI. RESULTS

A. RQ1: Faithfulness

TABLE VI
THE FAITHFULNESS CHECKS OF RQ1, EACH AGAINST A REFERENCE COMPUTED WITHOUT QUSLICE

Check	Subjects	Agree
trace equals the circuit Qiskit builds	our runnable programs, up to 1,842 operations	16 of 16
OpenQASM 2 front end's trace equals the circuit Qiskit loads	OpenQASM 2 files of the corpora that Qiskit accepts	1,127 of 1,127
the front end rejects a file only when Qiskit does	OpenQASM 2 files of the corpora that break a rule	85 of 85
the srcML markup gives back the file byte for byte	OpenQASM 2 files up to 20 MB	1,228 of 1,228
OpenQASM 3 export of a program gives the trace of its Python source	our programs	24 of 24
OpenQASM 3 export of a file gives the circuit Qiskit loaded	corpus files	1,117 of 1,117
OpenQASM 3 parse matches the official reference	grammar tests	35 of 35
trace, read without running, equals the prepared circuit	Bugs4Q bugs	7 of 8
slices equal the light cone of the built circuit	our runnable programs	15 of 16
line slices equal those recorded while running	Bugs4Q units	17 of 17
deleting what the slice leaves out keeps each qubit's state	device-loop programs, 33 qubits, six inputs	10 of 10
srcSlice's classical lines contain the reference's	1,610 qubits in 401 files	90%
	our programs and Bugs4Q	97%
	held-out, 1,831 qubits	86%

Table VI summarizes the checks, and the paragraphs below explain the two that do not agree in full. The static trace matches what Qiskit builds. On all 16 of our runnable programs, of up to 1,842 operations, the trace equals the circuit the program builds when it runs. The OpenQASM 2 front end's trace equals the circuit Qiskit loads on all 1,127 files of the corpora that Qiskit accepts, operation for operation with its bits and conditions, and the front end rejects the other 85, which Qiskit rejects too. Its markup gives back every file it marks up byte for byte. It also reads the 36 programs that exercise the rules of the grammar and the 8 example programs of the specification as Qiskit does, and rejects 21 invalid programs. The OpenQASM 3 front end parses all 35 reference programs of the language's official grammar tests into the statements of their reference parse trees and rejects 126 invalid statements of those tests, accepting none. Exported by Qiskit to OpenQASM 3, all 24 of our programs give the trace of their

Python source, device loops, branches and switches included, and all 1,117 corpus files give the circuit Qiskit loaded. Of the 21 example programs of the language, 20 yield a trace, 12 of them with statements outside the subset reported, and 1 calls a gate it never defines, which QuSlice reports. 74 of VeriQBench's 94 OpenQASM 3 files are written in an earlier draft of the language, with several qubits in one declaration and operands without commas, and are rejected. Read without being run, the Bugs4Q programs yield the circuit their ported bug prepares for 7 of 8 bugs, and the remaining one rewrites its circuit as a DAG, which QuSlice reports. The slices match the light cone of the built circuit on 15 of our programs. The remaining one nests one condition inside another, which the reference slicer on the circuit reads by the outer condition only. On 30 random product inputs, exact simulation finds the reference's slice wrong 29 times and QuSlice's never, so there QuSlice is the more precise of the two. On all 17 Bugs4Q units whose operations map to lines, QuSlice's line slices equal those recorded while the ported programs run.

Loops that run on the device are sliced soundly. In the 10 programs with device loops, branches and switches, deleting every operation outside a qubit's slice from the source leaves the state of each of the 33 qubits unchanged at six inputs under exact simulation, as Proposition 1 predicts, while slices that read each loop body once are wrong on seven of the 10 programs.

The classical statements are where the approach is least exact. srcSlice's lines contain those of the flow-sensitive reference for 90% of 1,610 qubits in 401 files, for 97% on our programs and Bugs4Q, and for 273 of 276 qubits on Bugs4Q alone, with recall 0.92 and precision 0.52 against the reference, in 53 ms per file in the median. Before the subscript and module-variable fixes of E4, containment was 82% and 85%, below the threshold of 95%, and the fixes were made after that result. On the 1,831 qubits of the 404 held-out programs, containment is 86%, close to the 88% of the sample of LintQ's corpus that exposed the gaps, so the fixes did not fit those files, but code from GitHub stays below the threshold. Most misses lie in functions that build a value and return it, and in containers filled by method calls, which srcSlice's forward slices do not link to the use. The extra lines are those any flow-insensitive slicer adds, such as the definition of an index in another loop, so QuSlice shows both sets of classical lines.

Answer to RQ1. Without running a program, QuSlice reconstructs the circuit it builds and slices it as the light cone of the built circuit does, device loops included. Its classical statements are complete for most qubits but miss values returned from functions in held-out code.

B. RQ2: Coverage

Of the 7,568 programs of LintQ's corpus, QuSlice traces a circuit in 4,182. Of the 3,381 that yield none, 2,765 never construct a circuit, such as helpers and scripts that read results, 58 load their circuits from a file or a library, and 558 build them in code the interpretation does not follow, as Fig. 4a shows. It traces 111 of the 151 Bugs4Q programs and 60 of

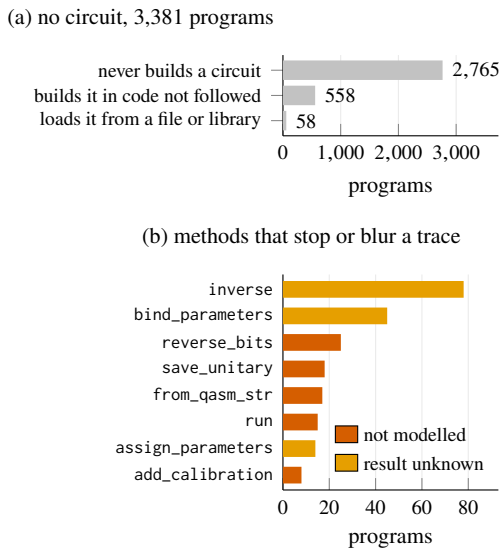


Fig. 4. Coverage on LintQ's corpus. (a) Why programs yield no circuit. (b) The methods that most often stop a trace, because the role table does not model them, or blur it, because the layer cannot know their result.

the 72 textbook programs, 103 of the Bugs4Q ones without any over-approximation. On current Qiskit, it traces 95% of the 82 Qiskit HumanEval programs that construct a circuit. Where tracing stops, the cause is mostly a method outside the role table, as Fig. 4b shows. 183 programs of LintQ's corpus call one, most often `reverse_bits`, a simulator instruction such as `save_unitary`, a circuit built from text with `from_qasm_str`, or `run`, and `prepare_state` stops three Qiskit HumanEval programs. Another 142 programs apply a method whose result the layer cannot know, most often `inverse` or `bind_parameters`, so the trace continues with that operation over-approximated. Adding such methods is the main way to widen coverage.

No program of Bugs4Q or LintQ's corpus crashes QuSlice.

Answer to RQ2. The approach slices most programs that build a circuit in code, and its failures are reported with a cause, most often a method it does not yet model.

C. RQ3: Cost and scalability

QuSlice slices the programs developers write in milliseconds and circuits of millions of operations in seconds once they are marked up, as Fig. 5 shows. On LintQ's corpus it slices the 4,182 programs that build a circuit, every qubit up to 64 of them, in 131 ms per program in the median and within 101 s for 99% of them, and 5 programs reach the limit of 120 s. A Bugs4Q program takes 33 ms in the median. srcML and srcSlice take most of the median, 32 and 57 ms, and the slices themselves 0.1 ms. On the 161 QASMBench circuits in valid OpenQASM 2, of up to 5.8 million operations and 433 qubits, slicing every qubit in one pass takes at most 10 s, on the largest circuit, after 12 minutes of reading, 532 s to produce its srcML and 171 s for the layer to read it. Markup runs at about 8,300 operations per second on the large circuits, a cost paid

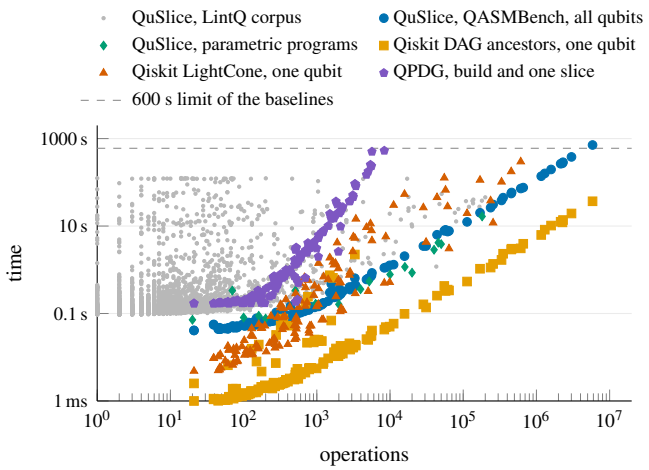


Fig. 5. Time of QuSlice against the operations of a trace. Grey points are the programs of LintQ's corpus, with every stage. The QASMBench points slice every qubit in one pass, producing and reading the srcML included, and the baselines slice one qubit of the same circuits, loading included. Baseline runs beyond their 600 s limit are left out.

TABLE VII
PREDICATES ON QUSLICE'S QUBIT PROFILES AT THE LINES OF LINTQ'S WARNINGS OF ITS FIVE QUBIT ANALYSES

Warnings	Count	Traced	Predicate holds
True positives, by LintQ's labels	142	139	139
False positives, by LintQ's labels	42	37	30
All warnings of these analyses	4,079	3,496	3,359

The analyses are operation after measurement, double measurement, oversized circuit, conditional without measurement and unmeasurable qubits.

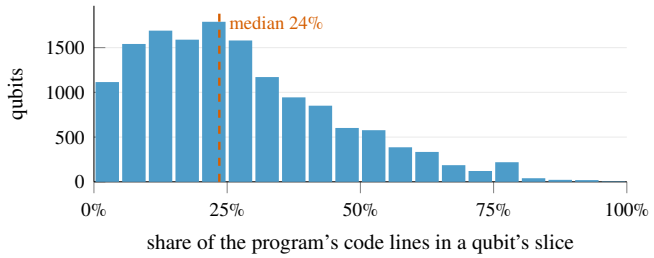


Fig. 6. The share of a program's code lines that a qubit's slice holds, over the 14,781 sliced qubits of LintQ's corpus.

once per file, as parsing with srcML is. The other 6 circuits are not valid OpenQASM 2, and Qiskit rejects them too. Qiskit's DAG slices one of its 97 qubits in 11 s after 26 s of loading, about 18 minutes for all of them. Qubit for qubit the two are close, one QuSlice slice taking 1.3 times Qiskit's in the median, so QuSlice's advantage on large circuits is the single pass over every qubit, which gives every slice sooner than Qiskit gives them one by one even with the markup counted. The QPDG reimplementation builds its graph within the limit for circuits of up to 8,355 operations, in 531 s, where QuSlice reads and slices the circuit in 1.11 s. Qiskit's `LightCone` pass slices one qubit of the circuit of 610,484 operations in 294 s, where QuSlice slices all 57 of its qubits in 1.2 s. On the parametric programs the source stays at about a dozen lines while the trace grows. The QFT on 600 qubits unrolls 181,200 operations from 11 code lines in 17 s, where running the program with Qiskit and slicing one qubit takes 1.5 s, the speed a static slicer gives up so that it never executes the program.

Answer to RQ3. The approach takes a fraction of a second per real program and scales to circuits of millions of operations, where producing their srcML takes minutes and its one pass over every qubit outpaces tools that slice one qubit of the built circuit.

D. RQ4: What a slice shows

A slice narrows what a developer reads. A qubit's slice holds 24% of a program's code lines in the median over the 14,781 sliced qubits of LintQ's corpus, together with the loop iterations that put each line there, and Fig. 6 shows that most slices hold less than half of the program. The profile shows two directions, and they differ. Over the 290 qubits of Bugs4Q, the 196 of the textbook and the 1,058 of 500 sampled files of LintQ's corpus, a qubit's slice and the forward slice of its first operation differ in length for 59%, 75% and 82% of the qubits, so what determines a qubit is seldom what it goes on to affect.

Slices answer a different question from a bug detector's, where to look rather than what is wrong, and the comparison with LintQ asks what the qubit view adds to its warnings. Table VII shows that the profiles contain what LintQ's qubit analyses check. Of the 142 warnings LintQ's authors labelled true positives, QuSlice traces 139, and a predicate on the profiles holds at the warned line of all of them. The predicates also hold on 30 of the 37 traced warnings the authors labelled false positives, so the profiles reproduce these analyses with their false alarms and are not a more precise detector. LintQ's other five analyses concern API use, such as a deprecated gate, and lie outside any slice. What they add is context. LintQ points to one line, and the slices of the qubits on that line give the 8 lines in the median that determine their state, out of 62 code lines in the file. LintQ's bound of ten loop iterations also matters. In 251 of the programs QuSlice traces, operations are created after the tenth iteration of a loop, 69% of all their operations.

Table VIII sets QuSlice beside the analyzers that read quantum source code. LintQ orders two operations exactly when they share a qubit, the relation a slice follows, but uses the order for adjacent patterns, and QChecker matches calls in the syntax tree with no model of qubits or order. On the examples of their own papers, transcribed as programs, the slices show each bug. In LintQ's Fig. 1, the slice of q_3 is empty, which is the oversized circuit, and the slice of q_0 holds the `ry` applied after its measurement. In QChecker's Fig. 5, the slice of q_2 holds the measurement of q_1 and the `cx` that follows it on q_1. QChecker's authors report that it cannot detect the Bugs4Q bugs that show only as wrong outputs [8]. Every Bugs4Q unit of RQ1 changes an output, and on 15 of the 15

TABLE VIII
QUSLICE BESIDE THE TOOL IT BUILDS ON AND THE STATIC ANALYZERS OF QUANTUM SOURCE CODE. TIMES ARE AS EACH PAPER REPORTS THEM, ON ITS OWN CORPUS AND MACHINE.

	srcSlice [4]	**QStatic** [9]	**LintQ** [7]	**QChecker** [8]	**QuSlice**
Input	C, C++, C#, Java	OpenQASM 3, hand-marked-up srcML	Qiskit	Qiskit	Qiskit, OpenQASM 2, OpenQASM 3
Representation	srcML	srcML	CodeQL database	Python AST	srcML, srcSlice, operation trace
Qubits	none	per-qubit order of operations	register and index	none	global index per operation, over-approximated when unknown
Loops	all, flow-insensitive	unrolled, no `while`	unrolled if the bound is known and at most 10	none	unrolled if the bound is known, with iterations, device loops repeated
Dependence	def-use of variables	none	same-qubit order	none	backward light cone, followed through measured bits
Slices	forward, of variables	none	none	none	backward and forward, of qubits, by line and iteration
Output	variable profiles	patterns, refactorings	warnings, 10 analyses	warnings, 8 detectors	slices, qubit profiles
Time per program	Linux kernel in 20 min	–	1.3 s	48.2 ms	131 ms in the median on LintQ's corpus

units whose faulty lines map to the source, the slice keeps the faulty line, although these programs are small enough that a slice holds most of their lines.

Answer to RQ4. A slice asks a developer to read about a quarter of a program, and its profile separates what determines a qubit from what the qubit affects. Beside a bug detector it shows the few lines that determine the qubits a warning names, and it keeps the faulty line of bugs that no pattern detects.

VII. DISCUSSION AND THREATS TO VALIDITY

Construct validity: Faithfulness is measured against the circuit Qiskit builds and the light cone of that circuit, so it shows that the approach computes what the rule defines, not that the rule suits every question a developer asks. The light cone is taken as the dependence relation, as compilers use it [26], [27]. The share of code lines a slice holds measures how much a developer is asked to read, not whether reading it helps, which needs a study with developers. The comparison with LintQ uses LintQ's own labels of its warnings.

Internal validity: The runnable programs that check the trace and the programs with device loops are our own, written while the tool was built, so the held-out sample of LintQ's corpus is the better estimate of how the classical statements fare on new code. The reference slicer on circuits is separate code from the tool but follows the same rule. Every analysis is exploratory and was logged with its changes before its results.

External validity: Most real programs come from LintQ's corpus and Bugs4Q, which predate Qiskit 1.0, and Qiskit HumanEval covers current releases with short programs. Qiskit is the only library the source front end reads. Times were measured on one machine, and the large circuits come from QASMBench.

VIII. CONCLUSION

Quantum programs can be sliced statically, on the source developers write. A quantum layer over srcML and srcSlice reads the quantum role of each call, reconstructs the circuit a program builds, and slices every qubit along the gates and measured bits that can affect it, loops on the device included, showing the result as lines, iterations and the classical statements behind them. On thousands of real programs it reconstructs the circuits Qiskit builds, slices them as their light cone does, takes a fraction of a second per program, and asks a developer to read about a quarter of a program. Widening the role table, linking values returned from functions, and a study of how developers use the slices are the next steps.

REFERENCES

[1] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, "Quantum computing with Qiskit," arXiv:2405.08810, 2024.

[2] J. Zappin, T. Stalnaker, O. Chaparro, and D. Poshyvanyk, "Challenges and practices in quantum software testing and debugging: Insights from practitioners," *ACM Transactions on Software Engineering and Methodology*, 2026, to appear.

[3] M. Weiser, "Program slicing," *IEEE Transactions on Software Engineering*, vol. SE-10, no. 4, pp. 352–357, 1984.

[4] H. W. Alomari, M. L. Collard, J. I. Maletic, N. Alhindawi, and O. Meqdadi, "srcSlice: very efficient and scalable forward static slicing," *Journal of Software: Evolution and Process*, vol. 26, no. 11, pp. 931–961, 2014.

[5] M. L. Collard, M. J. Decker, and J. I. Maletic, "srcML: An infrastructure for the exploration, analysis, and manipulation of source code: A tool demonstration," in *IEEE International Conference on Software Maintenance (ICSM)*, 2013, pp. 516–519.

[6] IBM Quantum, "LightCone transpiler pass, Qiskit SDK 2.4 documentation," https://eu-de.quantum.cloud.ibm.com/docs/en/api/qiskit/2.4/qiskit.transpiler.passes.LightCone, 2026, accessed 2026-10-29.

[7] M. Paltenghi and M. Pradel, "Analyzing quantum programs with LintQ: A static analysis framework for Qiskit," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 2144–2166, 2024.

[8] P. Zhao, X. Wu, Z. Li, and J. Zhao, "QChecker: Detecting bugs in quantum programs via static analysis," in *IEEE/ACM International Workshop on Quantum Software Engineering (Q-SE)*, 2023, pp. 50–57.

[9] J. A. C. Behler, A. F. Al-Ramadan, B. Baheri, M. L. Collard, Q. Guan, and J. I. Maletic, "Static analysis and transformation for quantum programming languages," *IEEE Software*, vol. 42, no. 5, pp. 58–64, 2025.

[10] J. A. C. Behler, A. F. Al-Ramadan, B. Baheri, Q. Guan, and J. I. Maletic, "Supporting static program analysis and transformation of quantum-based languages," in *IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2024, pp. 836–842.

[11] P. Zhao, Z. Miao, S. Lan, and J. Zhao, "Bugs4Q: A benchmark of existing bugs to enable controlled testing and debugging studies for quantum programs," *Journal of Systems and Software*, vol. 205, p. 111805, 2023.

[12] Qiskit Community, "The Qiskit textbook," https://github.com/qiskit-community/qiskit-textbook, 2024, accessed 2026-10-26.

[13] S. Vishwakarma, F. Harkins, S. Golecha, V. S. Bajpe, N. Dupuis, L. Buratti, D. Kremer, I. Faro, R. Puri, and J. Cruz-Benito, "Qiskit HumanEval: An evaluation benchmark for quantum code generative models," arXiv:2406.14712, 2024.

[14] A. Li, S. Stein, S. Krishnamoorthy, and J. Ang, "QASMBench: A low-level quantum benchmark suite for NISQ evaluation and simulation," *ACM Transactions on Quantum Computing*, vol. 4, no. 2, pp. 1–26, 2023.

[15] M. L. Collard, M. J. Decker, and J. I. Maletic, "Lightweight transformation and fact extraction with the srcML toolkit," in *IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2011, pp. 173–184.

[16] J. Ferrante, K. J. Ottenstein, and J. D. Warren, "The program dependence graph and its use in optimization," *ACM Transactions on Programming Languages and Systems*, vol. 9, no. 3, pp. 319–349, 1987.

[17] S. Horwitz, T. Reps, and D. Binkley, "Interprocedural slicing using dependence graphs," *ACM Transactions on Programming Languages and Systems*, vol. 12, no. 1, pp. 26–60, 1990.

[18] M. Kaul, A. Küchler, and C. Banse, "A uniform representation of classical and quantum source code for static code analysis," in *IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2023, pp. 1013–1019.

[19] P. K. Nayak, K. V. Kher, M. B. Chandra, M. V. P. Rao, and L. Zhang, "Q-PAC: Automated detection of quantum bug-fix patterns," arXiv:2311.17705, 2023.

[20] Q. Chen, R. Câmara, J. Campos, A. Souto, and I. Ahmed, "The Smelly Eight: An empirical study on the prevalence of code smells in quantum computing," in *IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023, pp. 358–370.

[21] S. Xia and J. Zhao, "Static entanglement analysis of quantum programs," in *IEEE/ACM International Workshop on Quantum Software Engineering (Q-SE)*, 2023, pp. 42–49.

[22] IBM Quantum, "Qiskit addon utilities: `slicing` module," https://quantum.cloud.ibm.com/docs/en/api/qiskit-addon-utils/slicing, 2026, accessed 2026-10-26.

[23] S. A. Metwalli and R. Van Meter, "A tool for debugging quantum circuits," in *IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2022, pp. 624–634.

[24] ——, "Testing and debugging quantum circuits," *IEEE Transactions on Quantum Engineering*, vol. 5, p. 2500415, 2024.

[25] H. Yu and J. Zhao, "The quantum program dependence graph and its uses in quantum software development," in *IEEE/ACM International Workshop on Quantum Software Engineering (Q-SE)*, 2025, pp. 48–55.

[26] P. Murali, N. M. Linke, M. Martonosi, A. Javadi-Abhari, N. H. Nguyen, and C. H. Alderete, "Full-stack, real-system quantum computer studies: Architectural comparisons and design insights," in *International Symposium on Computer Architecture (ISCA)*, 2019, pp. 527–540.

[27] M. Amy and V. Gheorghiu, "staq—A full-stack quantum processing toolkit," *Quantum Science and Technology*, vol. 5, no. 3, p. 034016, 2020.

[28] J. M. Murillo, J. Garcia-Alonso, E. Moguel, J. Barzen, F. Leymann, S. Ali, T. Yue, P. Arcaini, R. Pérez-Castillo, I. García-Rodríguez de Guzmán, M. Piattini, A. Ruiz-Cortés, A. Brogi, J. Zhao, A. Miranskyy, and M. Wimmer, "Quantum software engineering: Roadmap and challenges ahead," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 154:1–154:48, 2025.

[29] N. C. L. Ramalho, H. A. de Souza, and M. L. Chaim, "Testing and debugging quantum programs: The road to 2030," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, pp. 155:1–155:46, 2025.

[30] A. W. Cross, L. S. Bishop, J. A. Smolin, and J. M. Gambetta, "Open quantum assembly language," 2017.

[31] A. Cross, A. Javadi-Abhari, N. D. Beaudrap, L. S. Bishop, S. Heidel, C. A. Ryan, P. Sivarajah, J. Smolin, and J. M. Gambetta, "Openqasm 3: A broader and deeper quantum assembly language," *ACM Transactions on Quantum Computing*, 2022.

[32] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.

[33] K. Chen, W. Fang, J. Guan, X. Hong, M. Huang, J. Liu, Q. Wang, and M. Ying, "VeriQBench: A benchmark for multiple types of quantum circuits," arXiv:2206.10880, 2022.